CLAUDE CODE ULTIMATE GUIDE

# Claude Code Cheatsheet

Daily essentials for maximum productivity

Florian BRUNIAUX

July 2026

v1.0.0

Guide v3.41.1

# Table of contents

# 1 How to Install

```
npm install -g @anthropic-ai/claude-code # Verify installation which claude && claude --version
```

**Health Check:** `claude doctor && claude mcp list`

# 2 Essential Commands

| Command | Action |
|---|---|
| /help | Contextual help |
| /powerup | Interactive animated lessons teaching Claude Code features |
| /clear | Reset conversation |
| /compact | Free up context |
| /status | Session state + context usage |
| /context | Detailed token breakdown |
| /plan | Enter Plan Mode (no changes) |
| Approve the plan | Exit Plan Mode ( Shift+Tab to leave without approving) |
| /model | Switch model (sonnet/opus/opusplan) |
| /insights | Usage analytics + optimization report |
| /simplify | Cleanup-only review of changed code: reuse, simplification, efficiency |
| /batch | Large-scale refactors via 5-30 parallel worktree agents |
| /teleport | Teleport session from web |
| /tasks | Monitor background tasks |
| /remote-env | Configure cloud environment |
| /rc | Start remote control session (Research Preview, Pro/Max) |
| /mobile | Get Claude mobile app download links |
| /fast | Toggle fast mode (2.5x speed, 6x cost) |
| /voice | Toggle voice input (hold Space to speak) |
| /recap | Session context summary on return to a break |
| /exit ɡ [name] rompt] | [illegible] |

# 3 Keyboard Shortcuts

| Shortcut | Action |
|---|---|
| `Shift+Tab` | Cycle permission modes |
| `Esc × 2` | Rewind (undo) |
| `Ctrl+C` | Interrupt |
| `Ctrl+R` | Search command history |
| `Ctrl+L` | Clear screen (keeps context) |
| `Ctrl+B` | Background tasks |
| `Ctrl+F` | Kill all background agents (double press) |
| `Alt+T` | Toggle thinking on/off |
| `Space` (hold) | Voice input (requires `/voice`) |
| `Tab` | Autocomplete |
| `Shift+Enter` | New line |
| `Ctrl+D` | Exit |

**IDE Shortcuts**: VS Code `Alt+K` | JetBrains `Cmd+Option+K`

# 4 File References

`@path/to/file.ts` → Reference a file `@agent-name` → Call an agent `!shell-command` → Run shell command

# 5 Lesser-Known Features (But Official!)

| Feature | What It Does |
|---|---|
| **Tasks API** | Persistent task lists with dependencies |
| **Background Agents** | Sub-agents work while you code |
| **Agent Teams** | Multi-agent coordination (TeamCreate/SendMessage) |
| **Auto-Memories** | Automatic cross-session context capture |
| **Session Forking** | Rewind + create parallel timeline |
| **LSP Tool** | Code intelligence (go-to-def, refs), ~50ms vs 45s with grep |
| **Voice Mode** | Native voice input, free transcription |
| **Remote Control** | Control local session from phone/browser (Research Preview, Pro/Max) |
| **Cloud/Desktop Scheduled Tasks** | Machine-off scheduling via `/schedule` or Desktop app |
| **Skill Evals** | A/B testing and trigger tuning for custom skills |
| **Output Styles** | `/config` → Default, Explanatory, Learning modes |
| **Rules Templates** | Ready-to-use `.claude/rules/` templates (arch/quality/test/perf) |

**Pro tip**: These aren't "secrets", they're in the CHANGELOG. Read it!

# 6 Permission Modes

| Mode | Editing | Execution |
|---|---|---|
| Default | Asks | Asks |
| acceptEdits | Auto | Asks |
| Plan Mode | None | None |
| auto | Classifier decides | Classifier decides |
| dontAsk | Only if in allow rules | Only if in allow rules |
| bypassPermissions | Auto | Auto (CI/CD only) |

**Shift+Tab** to switch modes

# 7 Memory & Settings (2 levels)

| Level | Path | Scope | Git |
|---|---|---|---|
| **Project** | `.claude/` | Team | Yes |
| **Personal** | `~/.claude/` | You (all projects) | No |

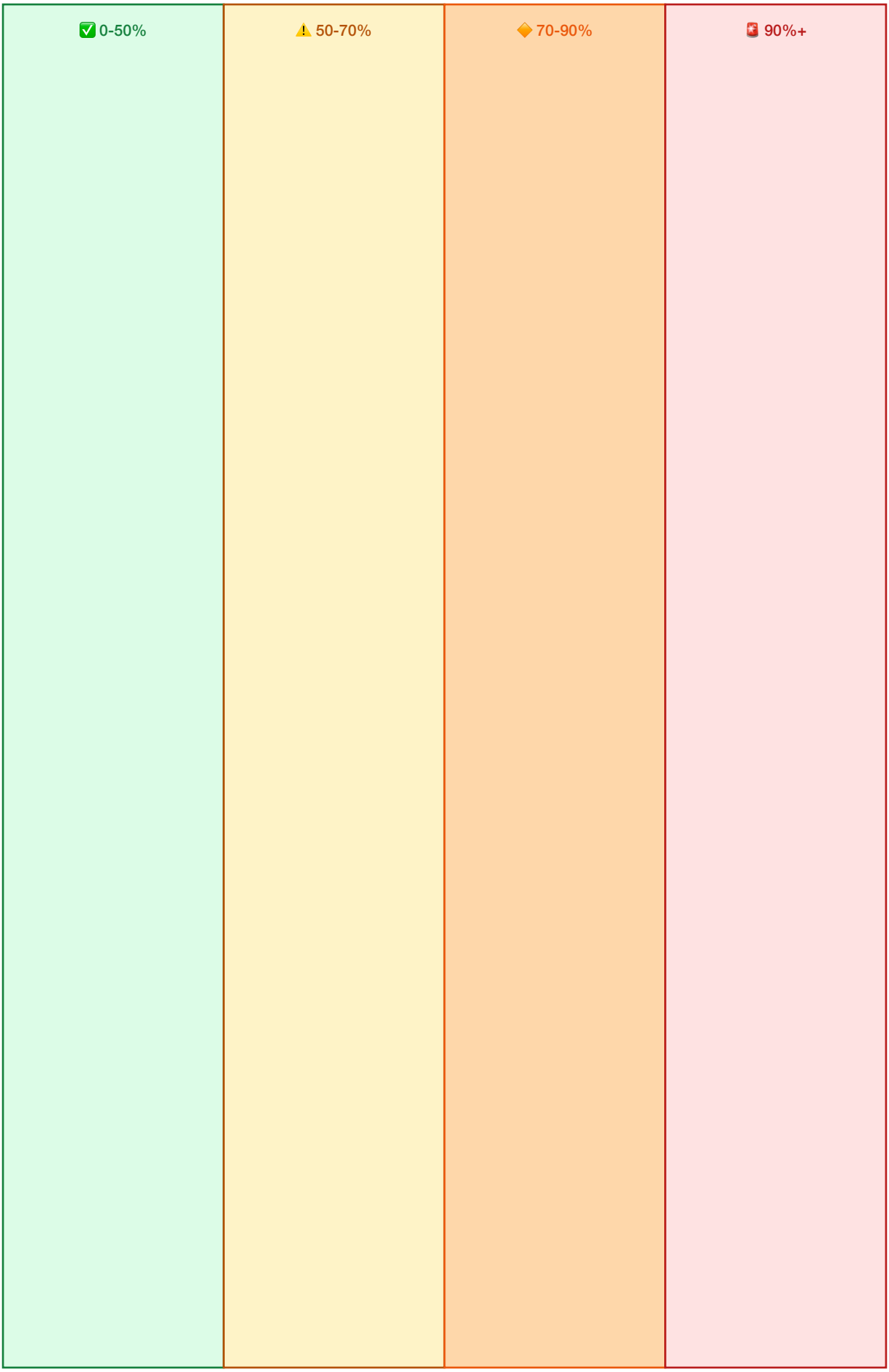
**Priority**: Project overrides Personal

## 7.1 .claude/ Folder Structure

```
.claude/
├── CLAUDE.md                # Local memory (gitignored)
├── settings.json            # Hooks (committed)
├── settings.local.json      # Permissions (not committed)
├── agents/                  # Custom agents
├── hooks/                   # Event scripts
├── rules/                   # Auto-loaded rules
│   ├── architecture-review.md
│   ├── code-quality-review.md
│   ├── test-review.md
│   └── performance-review.md
└── skills/                  # Slash commands + knowledge modules (unified)
```

# 8 Context Management (CRITICAL)

**Statusline:** `Model: Sonnet | Ctx: 89.5k | Ctx(u): 56.0%`


✅ 0-50%
⚠️ 50-70%
🔶 70-90%
🚨 90%+

**Watch** `Ctx(u):` → >70% = `/compact`, >85% = `/clear`

| Sign | Action |
|---|---|
| Short responses | `/compact` |
| Frequent forgetting | `/clear` |
| >70% context | `/compact` |
| Task complete | `/clear` |

## 8.1 Context Recovery Commands

| Command | Usage |
|---|---|
| `/compact` | Summarize and free context |
| `/clear` | Fresh start |
| `/rewind` | Undo recent changes |
| `claude -c` | Resume last session |
| `claude -r <id>` | Resume specific session |

# 9 Plan Mode & Thinking

| Feature | Activation | Usage |
|---|---|---|
| **Plan Mode** | `Shift+Tab × 2` or `/plan` | Explore without modifying |
| **OpusPlan** | `/model opusplan` | Opus for planning, Sonnet for execution |

**Opus 4.8**: Default effort is **xhigh**. Use `ultrathink` to force max effort for the next turn.

| Control | Action | Persistence |
|---|---|---|
| **Alt+T** | Toggle thinking on/off | Session |
| **/config** | Enable/disable globally | Permanent |
| `/effort` **slider** | low/medium/high/xhigh/max | Session |
| `effort` **param** | API only, per-request | Per-request |

**Cost tip**: For simple tasks, Alt+T to disable thinking → faster & cheaper.

# 10 Typical Workflow

1. Start session → claude
2. Check context → /status
3. Plan Mode → Shift+Tab × 2 (for complex tasks)
4. Describe task → Clear, specific prompt
5. Review changes → Always read the diff!
6. Accept/Reject → y/n
7. Verify → Run tests
8. Commit → When task complete
9. /compact → When context >70%

# 11 Quick Prompting Formula

```
WHAT: [Concrete deliverable]  WHERE: [File paths]  HOW: [Constraints, approach]  VERIFY: [Success criteria]
```

### Example:

```
Add input validation to the login form.  WHERE: src/components/LoginForm.tsx
HOW: Use Zod schema, show inline errors  VERIFY: Empty email shows error, invalid format shows error
```

# 12 MCP Servers

| Server | Purpose |
|---|---|
| **Serena** | Indexing + session memory + symbol search |
| **grepai** | Semantic search + call graph analysis |
| **Context7** | Library documentation |
| **Sequential** | Structured reasoning |
| **Playwright** | Browser automation |
| **Postgres** | Database queries |
| **doobidoo** | Semantic memory + Knowledge Graph |

**Serena memory:** `write_memory()` / `read_memory()` / `list_memories()`

Check status: `/mcp`

# 13 Search Tools Quick Reference

Quick decision: exact text → `rg` | exact name → `rg`/Serena | concept → grepai | structure → ast-grep

| Task | Tool |
|---|---|
| Find exact text | `rg "TODO"` |
| Find auth-related code | `grepai search "authentication"` |
| Who calls a function | `grepai trace callers "login"` |
| Get file structure | `serena get_symbols_overview` |

# 14 CLI Flags Quick Reference

| Flag | Usage |
|---|---|
| `-p "query"` | Non-interactive mode (CI/CD) |
| `-c` / `--continue` | Continue last session |
| `-r` / `--resume <id>` | Resume specific session |
| `--teleport` | Teleport session from web |
| `--model sonnet` | Change model |
| `--add-dir ../lib` | Allow access outside CWD |
| `--permission-mode plan` | Plan mode |
| `--worktree` / `-w` | Run in isolated git worktree |
| `--max-budget-usd 5.00` | Max API spend limit (print mode) |
| `--dangerously-skip-permissions` | Auto-accept (use carefully) |
| `--mcp-debug` | Debug MCP servers |
| `--allowedTools "Edit,Read"` | Whitelist tools |
| `--debug` | Debug output |

# 15 Anti-patterns

### ❌ Don’t

- Vague prompts
- Accept without reading
- Ignore warnings
- Skip permissions
- Negative constraints only

### ✅ Do

- Specify file + line with @references
- Read every diff
- Use /compact at 70%
- Never in production
- Provide alternatives

# 16 Cost Optimization

| Model | Use For | Cost |
|---|---|---|
| Haiku | Simple fixes, reviews | $ |
| Sonnet | Most development | $$ |
| Opus | Architecture, complex bugs | $$$ |
| OpusPlan | Plan (Opus) + Execute (Sonnet) | $$ |

# 17 Quick Decision Tree

Simple task → Just ask Claude
Complex task → Tasks API to plan first
Risky change → Plan Mode first
Repeating task → Create agent or command
Context full → /compact or /clear
Need docs → Use Context7 MCP
Deep analysis → Use Opus (thinking on by default)

## The Golden Rules

1. Always review diffs before accepting
2. Use /compact before context gets critical (>70%)
3. Be specific in requests (WHAT, WHERE, HOW, VERIFY)
4. Plan Mode first for complex/risky tasks
5. Create CLAUDE.md for every project
6. Commit frequently after each completed task
7. Know what's sent: prompts, files, MCP results → Anthropic

# 18 Common Issues Quick Fix

| Problem | Solution |
|---|---|
| “Command not found” | Check PATH, reinstall npm global |
| Context too high (>70%) | `/compact` immediately |
| Slow responses | `/compact` or `/clear` |
| MCP not working | `claude mcp list`, check config |
| Permission denied | Check `settings.local.json` |
| Hook blocking | Check hook exit code, review logic |

**Health Check**: `which claude && claude doctor && claude mcp list`

---

**Author**: Florian BRUNIAUX | Methode Aristote | Written with Claude

*Version 3.41.1 | July 2026*